

AWS/GCP 실전 마스터 2일차

1교시: 고가용성 및 로드 밸런싱 개념, ELB/Cloud Load Balancing 소개

개념 설명: 고가용성(High Availability)은 시스템이 장애 발생 시에도 서비스를 계속 제공할 수 있도록 여러 구성 요소를 중복 배치하는 설계 원칙입니다 ①. 이를 위해 여러 가용 영역(AZ)에 서버를 배포하고, 로드 밸런서를 통해 트래픽을 분산시킵니다. **로드 밸런서**는 사용자 요청을 여러 백엔드 서버에 고르게 분산시켜 특정 서버의 과부하를 방지하고 시스템 안정성을 높여줍니다 ② ③. AWS에서는 ELB(Elastic Load Balancer)를 사용하고, GCP에서는 Cloud Load Balancing을 사용해 전역(멀티리전) 또는 리전별로 부하를 분산시킬 수 있습니다. Cloud Load Balancing은 Google의 전역 프록시 기술을 활용해 여러 리전의 백엔드에 초당 수백만 건의 요청을 효율적으로 전달하며, 인스턴스의 장애를 감지해 자동으로 트래픽을 제외합니다 ③ ④.

[아키텍처 다이어그램 삽입]

실습 목표 및 시나리오:

- **목표:** ELB(ALB/NLB)와 GCP Cloud Load Balancer를 생성하고, 예제 웹 서버에 트래픽을 분산시켜 고가용성을 체험합니다.

- **배경 시나리오:** 스타트업 A사는 인기 웹사이트를 운영 중이며, 갑작스런 트래픽 증가에 대비해 서버를 다중 배치하고 로드 밸런서를 사용하여 서비스 다운타임을 최소화하려 합니다. 이 실습에서는 이를 위해 서버를 여러 대 띄우고 로드 밸런서로 묶어봅니다.

실습 절차:

1. **웹 서버 인스턴스 준비:** AWS EC2 인스턴스 2대(AMI: Amazon Linux 2)와 GCP Compute Engine 인스턴스 2대를 생성합니다. 각 서버에 간단한 웹 페이지("Hello from AWS" / "Hello from GCP")를 띄웁니다.

2. 로드 밸런서 생성:

- **AWS:** AWS Management Console 또는 AWS CLI를 통해 ALB(Application Load Balancer)나 NLB(Network Load Balancer)를 생성합니다.

- **GCP:** Google Cloud Console 또는 `gcloud` 명령어로 HTTP(S) 부하 분산기를 생성합니다.

3. **백엔드 그룹 구성:** 생성된 서버 인스턴스를 로드 밸런서의 대상(Target)으로 등록합니다. 보통 AWS ELB의 경우 **Target Group**에 인스턴스 등록, GCP는 **백엔드 서비스**에 인스턴스 그룹(Managed/Unmanaged Instance Group)을 연결합니다.

4. **보안 설정 확인:** 로드 밸런서와 인스턴스 간 통신을 위해 필요한 보안 그룹(AWS) 및 방화벽 규칙(GCP)을 설정합니다 (HTTP/HTTPS 포트 개방).

5. **테스트:** 로드 밸런서의 공개 DNS 또는 IP로 여러 차례 HTTP 요청을 보내고, 트래픽이 두 서버로 분산되는지 확인합니다.

실습 코드 예시:

- AWS CLI 예시:

```
# AWS: ELB 생성 및 인스턴스 등록 예시
aws elb create-load-balancer --load-balancer-name my-elb \
  --listeners "Protocol=HTTP,LoadBalancerPort=80,InstanceProtocol=HTTP,InstancePort=80" \
  --subnets subnet-abcdef12 subnet-345678gh --security-groups sg-01234abc
aws elb register-instances-with-load-balancer --load-balancer-name my-elb \
  --instances i-0123456789abcdef0 i-0fedcba9876543210
```

- GCP gcloud 예시:

GCP: HTTP 부하 분산기 생성 예시

```
gcloud compute instance-groups unmanaged create my-igm --instances web-1,web-2 --zone us-central1-a
gcloud compute health-checks create http my-hc --port 80
gcloud compute backend-services create my-backend --protocol HTTP --health-checks my-hc --global
gcloud compute backend-services add-backend my-backend --instance-group my-igm --instance-group-zone us-central1-a --global
gcloud compute url-maps create my-map --default-service my-backend
gcloud compute target-http-proxies create my-proxy --url-map my-map
gcloud compute forwarding-rules create my-fw --address 0.0.0.0 --global --target-http-proxy my-proxy --ports 80
```

실습 결과 예측:

- 로드 밸런서(DNS 또는 IP)에 접속하면 두 인스턴스가 번갈아 “Hello” 페이지를 응답합니다. 각 서버의 응답 비율이 약 50:50으로 분산되어 표시됩니다 ⁵.
- 헬스체크가 자동 적용되어, 특정 인스턴스가 응답하지 않으면 로드 밸런서가 다른 인스턴스로 트래픽을 보냅니다.
- AWS ELB 생성 시 반환된 DNS명과 예상 응답 예시는 다음과 같습니다: ⁵

DNSName: my-load-balancer-1234567890.us-west-2.elb.amazonaws.com

[스크린샷 예시 삽입] (로드 밸런서 DNS 출력 화면 예시)

요약 및 확인:

- **혼자 해보기:** ELB/NLB 대신 Application Load Balancer(ALB)를 사용해보고, HTTPS(SSL) 리스너를 추가 설정해 봅니다.
- **퀴즈:** 로드 밸런서가 인스턴스를 고르게 선택하는 대표적인 알고리즘 3가지를 적어보세요 (예: Round Robin, Least Connections, Weighted 등).
- **체크리스트:** - 인스턴스 2대 이상이 생성되었나요? - 각각에 웹서비스가 정상 동작하나요? - ELB/CLB가 만들어졌고 인스턴스가 대상(Target)으로 등록되었나요? - 로드 밸런서의 IP/DNS를 통해 서비스가 정상 분산 응답되나요?

2교시: 오토 스케일링 기본 개념 및 실습 (AWS EC2 ASG, GCP MIG)

개념 설명: 오토스케일링(Auto Scaling)은 클라우드의 핵심 기능으로, 서버 부하나 트래픽 변화에 따라 필요한 인스턴스 수를 **자동으로 증감**시켜 주는 기술입니다 ⁶ ⁷. 예를 들어, 한정된 시간 동안 갑자기 접속자가 많아지면 인스턴스를 추가(Scale Out)하고 트래픽이 줄면 인스턴스를 줄여(Scale In) 비용 효율과 성능을 동시에 높입니다 ⁸. AWS에서는 **Auto Scaling Group(ASG)**으로, GCP에서는 **Managed Instance Group(MIG)**으로 구현하며, CPU 사용률이나 부하 분산 서비스의 요청률 같은 메트릭에 따라 자동으로 조정합니다 ⁹ ⁷. ASG/MIG는 지정한 최소·최대 인스턴스 수 범위 내에서 필요한 만큼 인스턴스를 유지하며, 여러 AZ에 분산할 수도 있습니다.

[아키텍처 다이어그램 삽입]

실습 목표 및 시나리오:

- **목표:** AWS ASG와 GCP MIG를 구성하여 오토스케일링이 작동하는지 확인합니다.
- **배경 시나리오:** 웹 서비스에 갑작스러운 접속 폭주가 예상되어, 트래픽 증가 시 자동으로 인스턴스가 늘어나도록 설정하려고 합니다. 이 시간 동안 안정적으로 처리할 수 있는 서버를 자동으로 확보해 비용도 절약할 수 있습니다.

실습 절차:

1. **인스턴스 템플릿/설정 준비:** 먼저 AWS에 EC2 인스턴스용 Launch Configuration/Template을, GCP에 Instance Template을 생성합니다. 템플릿에는 앞서 만든 웹 서버 설정(예: 스타트업 스크립트 포함)을 정의합니다.
2. **오토 스케일링 그룹 생성:**
 - **AWS ASG:** 콘솔 또는 CLI로 Auto Scaling Group을 생성하여, 최소(min), 최대(max), 원하는(desired) 인스턴스 수를 설정합니다(예: min=1, max=3, desired=1).
 - **GCP MIG:** `gcloud` 또는 콘솔에서 Managed Instance Group을 생성하여, 인스턴스 템플릿과 크기(size)를 지정합니다. 영역(Zone) 단위와 리전 단위 MIG 중 선택 가능합니다.
3. **스케일링 정책 설정:** CPU 사용률을 예로 들어, 특정 임계치(예: 평균 CPU 60% 이상) 도달 시 인스턴스 1대 추가, 30% 이하로 내려가면 1대 제거하도록 정책을 만듭니다.
4. **트래픽 생성:** 웹 서버에 부하를 걸어 인스턴스 수가 증가하는지 확인합니다(예: Apache Bench 등 부하 툴 사용).
5. **스케일 인/아웃 확인:** 부하 제거 후 인스턴스 수가 줄어드는지 확인합니다.

실습 코드 예시:

- AWS CLI 예시:

```
# AWS: Launch Configuration 생성
aws autoscaling create-launch-configuration --launch-configuration-name my-launch-config \
--image-id ami-0123456789abcdef0 --instance-type t2.micro \
--security-groups sg-01234abc

# AWS: Auto Scaling Group 생성 (min=1, max=3)
aws autoscaling create-auto-scaling-group --auto-scaling-group-name my-asg \
--launch-configuration-name my-launch-config \
--min-size 1 --max-size 3 --desired-capacity 1 \
--vpc-zone-identifier subnet-abcdef12
```

- GCP gcloud 예시:

```
# GCP: 인스턴스 템플릿 생성
gcloud compute instance-templates create my-template \
--machine-type=e2-micro --image-family=debian-11 \
--image-project=debian-cloud

# GCP: 관리형 인스턴스 그룹(MIG) 생성 (size=1)
gcloud compute instance-groups managed create my-mig \
--base-instance-name my-instance \
--template my-template --size 1 --zone us-central1-a
```

실습 결과 예측:

- 설정된 메트릭(예: CPU 60%) 도달 시 AWS ASG와 GCP MIG가 자동으로 인스턴스를 추가합니다. 인스턴스 수가 예를 들어 1→2로 늘어나고, 실제로 콘솔에서 생성 로그가 확인됩니다. 부하가 사라지면 다시 2→1로 감소합니다.
- AWS CLI로 확인 시, `describe-auto-scaling-groups` 명령어로 현재 `DesiredCapacity`와 `Instances` 수가 변경된 것을 볼 수 있습니다.
- GCP에서는 `gcloud compute instance-groups managed describe my-mig` 명령어로 크기 변화 및 인스턴스 상태를 확인할 수 있습니다. [스크린샷 예시 삽입] (인스턴스 그룹 크기 변화를 보여주는 콘솔 화면)

요약 및 확인:

- **혼자 해보기:** 다른 메트릭(예: HTTP 요청 수, 네트워크 사용률 등)으로 스케일링 정책을 실험해 보세요. 설정한 정책이 작동하는 임계값을 조정해 보십시오.
- **퀴즈:** Auto Scaling의 스케일 아웃과 스케일 인이 의미하는 바를 설명해 보세요.
- **체크리스트:**
 - ASG/MIG가 생성되어 최소/최대 용량이 올바르게 설정되었나요?
 - 트래픽 부하를 걸었을 때 인스턴스가 증가하나요?
 - 부하 해제 후 인스턴스 수가 줄어드나요?
 - 콘솔/CLI로 현재 인스턴스 상태와 원하는(desired) 수를 확인했나요?

3교시: 로드 밸런서 + 오토스케일링 연동 및 상태 점검

개념 설명: 고가용성을 극대화하기 위해 로드 밸런서와 오토스케일링을 함께 사용합니다. Auto Scaling Group(ASG) 또는 Managed Instance Group(MIG)을 로드 밸런서 백엔드로 연결하면, 인스턴스 확장 시 자동으로 트래픽 분산 대상에 추가되고, 축소 시 제거됩니다. 또한 **상태 점검(Health Check)**이 핵심입니다. AWS ASG는 기본적으로 EC2 인스턴스 상태 확인을 수행하며, ELB 상태 확인을 활성화하면 ELB가 비정상이라고 판단한 인스턴스도 교체합니다 ¹⁰

¹¹ . GCP에서는 부하 분산기의 헬스 체크가 비정상 인스턴스를 감지하고 트래픽 전달을 중단하며, MIG는 이를 감지하여 인스턴스를 자동으로 복구(Healing)할 수 있습니다 ¹² ¹³ . 즉, 상태 점검이 실패한 인스턴스는 ASG/MIG에 의해 새로운 인스턴스로 대체되어 **자가 복구(Self-Healing)**가 이루어집니다.

[아키텍처 다이어그램 삽입]

실습 목표 및 시나리오:

- **목표:** 로드 밸런서와 ASG/MIG를 연동하고 상태 점검이 동작하는지 확인합니다.
- **배경 시나리오:** 스타트업 B사는 웹 서버 인스턴스를 ASG/MIG로 운영 중이며, ELB/CLB를 통해 분산 처리합니다. 이제 인스턴스 장애가 발생해도 서비스가 계속 유지되도록 설정하고자 합니다.

실습 절차:

1. 로드 밸런서에 인스턴스 그룹 연결:

- **AWS:** 이전 ASG를 편집하여 기존 ELB 또는 Target Group을 연결합니다. 또한 ASG 설정에서 **ELB 상태 점검 사용(Enable ELB Health Checks)** 옵션을 켭니다 ¹⁰ .
- **GCP:** 부하 분산기의 백엔드 서비스에 MIG를 추가하고, 이미 생성된 헬스 체크를 연동합니다.

2. 건전성 확인 설정:

- ASG/MIG의 **상태 확인 유예 기간(Health Check Grace Period)**을 적절히 설정하여, 인스턴스 시작 초기 부팅시간 등을 보장합니다.

3. 인스턴스 장애 시뮬레이션: 특정 인스턴스를 강제 종료 또는 네트워크 장애로 인식시킵니다. (예: AWS EC2 인스턴스 종료, GCP 인스턴스 강제 삭제)

4. 동작 확인:

- ELB/CLB 콘솔에서 해당 인스턴스의 상태가 `OutOfService`로 표시되고, 곧바로 ASG/MIG에서 새로운 인스턴스가 시작되는지 확인합니다 ¹¹ ¹² .
- 트래픽이 정상 인스턴스로 자동 라우팅되어 웹 서비스가 멈추지 않음을 확인합니다.

실습 코드 예시:

- AWS CLI 예시:

```
# AWS: ELB 대상 그룹에 ASG 연결 (CLI)
aws autoscaling attach-load-balancers --auto-scaling-group-name my-asg \
  --load-balancer-names my-elb
# 상태 점검 유예 기간 설정(예: 120초)
```

```
aws autoscaling update-auto-scaling-group --auto-scaling-group-name my-asg \
--health-check-grace-period 120 --health-check-type ELB
```

- GCP gcloud 예시:

```
# GCP: Managed Instance Group에 대상 풀(Target Pool) 추가 (네트워크 LB용 예시)
gcloud compute instance-groups managed set-target-pools my-mig \
--target-pools my-target-pool --zone us-central1-a
# 또는 HTTP(S) LB일 경우 백엔드 서비스에 자동으로 추가됨.
```

실습 결과 예측:

- 장애가 발생한 인스턴스는 ELB/CLB에서 Unhealthy 상태로 표시되고 곧 제거됩니다. AWS ASG는 새로운 인스턴스를 자동으로 추가하여 지정된 용량을 유지하며, GCP MIG도 헬스 체크 실패 후 복구 인스턴스를 생성합니다 ¹¹ ¹² .
- 서비스는 정상 인스턴스로만 요청을 받아 처리되며 사용자 관점에서는 다운타임 없이 동작합니다.
- **예상 콘솔 출력:** AWS ELB Target Group에서 Healthy → Unhealthy 전환 메시지, AWS ASG 또는 GCP MIG 로그에서 "Created a new instance to replace unhealthy instance" 와 같은 로그가 나타납니다. [스크린샷 예시 삽입] (ASG 복구 로그 예시)

요약 및 확인:

- **혼자 해보기:** 네트워크 장애 대신 애플리케이션 프로세스 내부 에러를 유발하여(예: 포트 비활성) 상태 점검 실패 시 복구를 유도해 보세요.
- **퀴즈:** ASG와 ELB의 헬스 체크는 어떤 차이가 있나요? AWS에서 ELB 상태 체크를 활성화하면 어떤 이점이 있나요?
- **체크리스트:**
 - ASG/MIG가 로드 밸런서에 정상적으로 연결되었나요?
 - 헬스 체크(상태 점검) 기능이 활성화되었나요?
 - 인스턴스 강제 종료 시 새로운 인스턴스가 기동되었나요?
 - 로드 밸런서에서 비정상 인스턴스가 자동으로 제외되었나요?

4교시: 장애 시뮬레이션 및 복구 실습

개념 설명: 실제 운영 환경에서는 서버 장애나 장애 이벤트를 대비해야 합니다. ASG/MIG는 **자가 치유(Self-Healing)** 기능을 제공합니다 ¹¹ ¹² . 즉, 인스턴스가 비정상 상태에 빠지면 자동으로 교체하여 항상 지정된 수의 정상 인스턴스를 유지합니다. 이는 ASG/MIG가 백그라운드에서 인스턴스 상태를 지속적으로 모니터링하고, 장애가 감지되면 **새로운 인스턴스를 시작하여 교체**하는 방식입니다 ¹¹ . 이를 통해 단일 인스턴스 장애에도 애플리케이션 전체는 계속 동작할 수 있습니다. 이 시간을 대화형으로 실습하면서, 장애 발생 시의 복구 과정을 확인합니다.

[아키텍처 다이어그램 삽입]

실습 목표 및 시나리오:

- **목표:** 의도적으로 인스턴스 장애를 발생시켜 AWS ASG와 GCP MIG가 이를 감지하고 자동 복구하는 과정을 관찰합니다.
- **배경 시나리오:** 서비스 운영 중 특정 서버가 다운되어도 자동으로 서버를 교체하여 서비스 연속성을 유지해야 합니다. 이 실습에서는 서버 장애를 시뮬레이션하고 복구 과정을 확인합니다.

실습 절차:

1. 장애 시뮬레이션:

- **AWS:** EC2 콘솔이나 `aws ec2 terminate-instances --instance-ids <ID>` 명령으로 ASG의 인스턴스 하나를 강제 종료합니다.
- **GCP:** GCP Console이나 `gcloud compute instances delete <INSTANCE_NAME> --zone=<ZONE>`

로 인스턴스 하나를 삭제합니다.

2. 복구 확인:

- ASG/MIG가 즉시 새로운 인스턴스를 시작하여 지정된 용량을 복구하는지 확인합니다. (AWS:

```
aws autoscaling describe-auto-scaling-groups, GCP:
gcloud compute instance-groups managed list-instances )
```

- 로드 밸런서에서 비정상 인스턴스가 제거되고, 새 인스턴스가 백엔드로 등록되어 트래픽을 수신하는지 검증합니다.

3. 장애 유형 다양화: 디스크 오류, 애플리케이션 크래시 등 다양한 장애를 유발해 보며 ASG/MIG 동작을 관찰합니다.

실습 코드 예시:

- AWS CLI 예시:

```
# AWS: ASG 인스턴스 중 하나 종료(대체 인스턴스 생성 유도)
INSTANCE_ID=$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-name my-asg --query "AutoScalingGroups[0].Instances[0].InstanceId" \
  --output text)
aws ec2 terminate-instances --instance-ids $INSTANCE_ID
```

- GCP gcloud 예시:

```
# GCP: MIG의 VM 하나 삭제
gcloud compute instances delete my-instance-0001 --zone us-central1-a --quiet
```

실습 결과 예측:

- 인스턴스 종료/삭제 직후, ASG/MIG 콘솔이나 로그에 “Unhealthy instance”, “Terminating instance”, “Launching a new instance to replace” 등의 메시지가 나타납니다. AWS는 종료된 인스턴스를 Terminating 으로 표시하고 새로운 인스턴스 ID로 InService 상태를 복원합니다. GCP MIG도 유사하게 새 인스턴스를 생성하여 RUNNING 상태로 전환합니다.

- 사용자 요청은 새로운 인스턴스로 장애 없이 지속 처리되며, 복구가 완료되면 ASG/MIG의 인스턴스 개수가 초기 상태로 회복됩니다 ¹¹ ¹². [스크린샷 예시 삽입] (ASG 로그 또는 콘솔에서 인스턴스 갱신이 이루어지는 모습 예시)

요약 및 확인:

- **혼자 해보기:** 장애가 발생한 상태에서 어떻게 알림을 설정할 수 있을지 고민해 보세요. 예를 들어 CloudWatch/Monitoring을 활용한 알람 설정을 추가해 보십시오.

- **퀴즈:** ASG/MIG가 비정상 인스턴스를 교체할 때 어떤 순서로 진행될까요? (예: 종료→새 인스턴스 시작 순서)

- 체크리스트:

- 강제로 인스턴스를 종료했나요?
- ASG/MIG에서 새로운 인스턴스가 생성되었나요?
- 로드 밸런서에 새로운 인스턴스가 정상 등록되었나요?
- 서비스는 장애 중에도 계속 응답하나요?

참고: 본 과정에서는 다수의 비전공자와 1인 기업 대표, 대학생, 스타트업 엔지니어도 쉽게 이해할 수 있도록 개념 설명과 단계별 실습을 충실히 다루었습니다. AWS와 GCP 환경 모두 손쉽게 실습할 수 있도록 예시 코드를 제공하였으니, 실습 환경에서 직접 실행해 보며 학습 효과를 높이시기 바랍니다 ² ¹¹.

1 2 4 고가용성(High Availability)을 위한 로드 밸런싱과 스케일링

<https://chanmin-tstory.tistory.com/entry/%EA%B3%A0%EA%B0%80%EC%9A%A9%EC%84%B1High-Availability%EC%9D%84-%EC%9C%84%ED%95%9C-%EB%A1%9C%EB%93%9C-%EB%B0%B8%EB%9F%B0%EC%8B%B1%EA%B3%BC-%EC%8A%A4%EC%BC%80%EC%9D%BC%EB%A7%81>

3 Cloud Load Balancing 개요 | Google Cloud

<https://cloud.google.com/load-balancing/docs/load-balancing-overview?hl=ko>

5 create-load-balancer — AWS CLI 2.28.6 Command Reference

<https://docs.aws.amazon.com/cli/latest/reference/elb/create-load-balancer.html>

6 8 9 [AWS] Auto Scaling 개념 원리 & 사용 세팅 정리

<https://inpa.tistory.com/entry/AWS-%F0%9F%93%9A-EC2-%EC%98%A4%ED%86%A0-%EC%8A%A4%EC%BC%80%EC%9D%BC%EB%A7%81-ELB-%EB%A1%9C%EB%93%9C-%EB%B0%B8%EB%9F%B0%EC%84%9C-%EA%B0%9C%EB%85%90-%EA%B5%AC%EC%B6%95-%EC%84%B8%ED%8C%85-%F0%9F%92%AF-%EC%A0%95%EB%A6%AC>

7 12 [GCP]How to Create Managed Instance Group (MIG)

<https://dewble.tistory.com/entry/how-to-create-managed-instance-group-mig-in-gcp>

10 Elastic Load Balancing 로드 밸런서를 Auto Scaling 그룹에 연결 - Amazon EC2 Auto Scaling

https://docs.aws.amazon.com/ko_kr/autoscaling/ec2/userguide/attach-load-balancer-asg.html

11 Health checks for instances in an Auto Scaling group - Amazon EC2 Auto Scaling

<https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-health-checks.html>

13 상태 점검 사용 | Load Balancing | Google Cloud

<https://cloud.google.com/load-balancing/docs/health-checks?hl=ko>